

CESAR SCHOOL
CIÊNCIA DA COMPUTAÇÃO
TEORIA DA COMPUTAÇÃO

ALGORITMO DE DIJKSTRA

Análise teórica e experimental do caminho mínimo:
implementações em Python e JavaScript

Lucas Emery
Pedro Ferraz
Gustavo Laporte
Luis Berard

Prof. Daniel Bezerra

Recife
2026

SUMÁRIO

1 Introdução	3
2 O Algoritmo de Dijkstra	4
2.1 Definição e objetivo	4
2.2 Funcionamento passo a passo	4
2.3 Estrutura de dados: fila de prioridade (min-heap)	4
2.4 Pseudocódigo	5
2.5 Complexidade computacional (O , Ω e Θ)	5
3 Metodologia Experimental	7
3.1 Implementações	7
3.2 Cenários de teste e geração das entradas	7
3.3 Tamanhos avaliados	7
3.4 Protocolo de medição	7
3.5 Ambiente de execução	8
3.6 Resumo dos cenários executados	8
4 Resultados e Análise dos Gráficos	10
4.1 Curvas de tempo por cenário	10
4.2 Dispersão e estabilidade das medições	15
4.3 Comparação entre Python e JavaScript	20
4.4 Validação do modelo teórico	24
5 Aplicabilidade: Eficiência e Limitações	31
5.1 Contextos de eficiência e aplicações	31
5.2 Limitações	31
6 Reflexão Teórica: Classes de Complexidade	32
6.1 O problema pertence à classe P?	32
6.2 Existe uma versão NP do problema?	32
6.3 Problemas semelhantes NP-completos	32
7 Execução do Projeto	34
7.1 Pré-requisitos	34
7.2 Executando os benchmarks	34
7.3 Gerando a análise e os gráficos	34
7.4 Gerando este relatório em PDF	34
8 Conclusão	35
REFERÊNCIAS	36

1 INTRODUÇÃO

O problema do caminho mínimo é um dos mais clássicos da Teoria da Computação e da área de grafos. Dado um grafo ponderado, deseja-se determinar o menor custo para ir de um vértice de origem até os demais vértices. Esse problema modela situações reais como roteamento em redes de computadores, sistemas de navegação (GPS), logística e jogos. Entre as soluções mais conhecidas está o algoritmo de Dijkstra, proposto por Edsger W. Dijkstra em 1959.

Este trabalho tem como objetivo estudar o algoritmo de Dijkstra sob duas perspectivas complementares. Na perspectiva teórica, descreve-se o que o algoritmo faz, como funciona passo a passo, qual a estrutura de dados empregada e qual a sua complexidade assintótica. Na perspectiva experimental, o algoritmo foi implementado em duas linguagens (Python e JavaScript) e submetido a um conjunto de experimentos controlados, com o objetivo de medir o tempo de execução em diferentes cenários e validar empiricamente o comportamento previsto pela teoria.

O relatório está organizado da seguinte forma. A Seção 2 apresenta a fundamentação teórica do algoritmo. A Seção 3 detalha a metodologia experimental, incluindo os casos de teste, a forma de geração das entradas, os tamanhos avaliados, o ambiente de execução e o protocolo de medição. A Seção 4 discute os resultados, analisando individualmente cada gráfico gerado. A Seção 5 aborda a aplicabilidade do algoritmo, com seus contextos de eficiência e limitações. A Seção 6 traz a reflexão teórica sobre as classes de complexidade (P e NP). A Seção 7 explica como executar os códigos do projeto e a Seção 8 apresenta as conclusões.

2 O ALGORITMO DE DIJKSTRA

2.1 Definição e objetivo

O algoritmo de Dijkstra resolve o problema do caminho mínimo de origem única (single-source shortest path) em grafos com pesos não negativos nas arestas. Partindo de um vértice de origem s , ele calcula a menor distância de s até todos os demais vértices do grafo. A restrição de pesos não negativos é essencial: ela garante que, uma vez que a menor distância de um vértice é determinada, ela não poderá ser melhorada posteriormente — propriedade que torna o algoritmo correto e eficiente.

Trata-se de um algoritmo guloso (greedy): a cada passo, ele escolhe o vértice ainda não processado que possui a menor distância conhecida até o momento, fixa essa distância como definitiva e tenta melhorar (relaxar) as distâncias de seus vizinhos.

2.2 Funcionamento passo a passo

O funcionamento do algoritmo pode ser resumido nos seguintes passos:

- Inicialização: a distância da origem para si mesma é definida como 0 e a distância para todos os outros vértices como infinito. A origem é inserida em uma fila de prioridade.
- Extração do mínimo: remove-se da fila o vértice u com a menor distância conhecida. Essa distância passa a ser considerada definitiva.
- Relaxamento: para cada vizinho v de u , verifica-se se o caminho que passa por u é mais curto, isto é, se $\text{dist}[u] + \text{peso}(u, v) < \text{dist}[v]$. Em caso afirmativo, $\text{dist}[v]$ é atualizada e v é reinserido na fila com a nova prioridade.
- Repetição: os passos de extração e relaxamento se repetem até que a fila fique vazia, momento em que todas as menores distâncias já foram calculadas.

Uma observação importante de implementação é o tratamento de entradas obsoletas (stale) na fila: como um vértice pode ser inserido várias vezes com distâncias diferentes, ao extraí-lo verificamos se a distância retirada é maior que a já registrada; se for, descartamos a entrada. Essa técnica é chamada de lazy deletion e evita o custo de remover elementos do meio da fila.

2.3 Estrutura de dados: fila de prioridade (min-heap)

A eficiência do algoritmo depende fortemente da estrutura usada para escolher, a cada passo, o vértice de menor distância. Neste trabalho utilizou-se uma fila de prioridade implementada como heap binário de mínimo (min-heap). O heap garante as operações de inserção e de remoção do menor elemento em tempo $O(\log V)$, em que V é o número de vértices.

Na implementação em Python, foi usado o módulo padrão `heapq`. Na implementação em JavaScript, como a linguagem não oferece uma fila de prioridade nativa, foi desenvolvida uma classe `MinHeap` própria, com os métodos de subida (`bubble up`) e descida (`sink down`) que mantêm a propriedade do heap.

2.4 Pseudocódigo

A seguir, o pseudocódigo do algoritmo implementado:

```
funcao Dijkstra(grafo, origem):
    dist[v] <- infinito para todo vertice v
    dist[origem] <- 0
    fila <- min-heap contendo (0, origem)

    enquanto fila nao vazia:
        (d, u) <- extrai_minimo(fila)
        se d > dist[u]:          # entrada obsoleta
            continua
        para cada (v, peso) em vizinhos(u):
            nd <- d + peso
            se nd < dist[v]:    # relaxamento
                dist[v] <- nd
                insere(fila, (nd, v))

    retorna dist
```

2.5 Complexidade computacional (O , Ω e Θ)

A análise de complexidade descreve como o tempo de execução cresce em função do tamanho da entrada. Para caracterizá-la de forma completa, utilizam-se três notações assintóticas: o limite superior Big-O (O), o limite inferior Big-Ômega (Ω) e o limite justo Big-Theta (Θ), este último válido quando os limites superior e inferior coincidem.

Considerando a implementação com fila de prioridade baseada em heap binário, cada uma das V extrações do mínimo custa $O(\log V)$ e cada uma das E arestas pode gerar uma inserção de $O(\log V)$. A inicialização do vetor de distâncias custa $\Theta(V)$. Com isso obtêm-se os seguintes limites:

- Limite superior — $O((V + E) \log V)$: no máximo, são feitas V extrações e E relaxamentos, cada qual com custo logarítmico. Nenhuma execução ultrapassa esse limite, independentemente do formato do grafo.
- Limite inferior — $\Omega(V + E)$: o algoritmo precisa, obrigatoriamente, inicializar as V distâncias e examinar todas as E arestas pelo menos uma vez para garantir que nenhuma distância pode ser melhorada. Portanto, nenhuma execução é mais rápida que linear no tamanho do grafo.

- Limite justo — $\Theta((V + E) \log V)$: nos grafos densos (pior caso), em que praticamente todas as arestas provocam relaxamentos com inserção no heap, os limites superior e inferior coincidem na mesma ordem de grandeza, e o tempo é descrito de forma justa por $\Theta((V + E) \log V)$.

É importante observar que os limites Ω e Θ dependem do conjunto de entradas considerado. No melhor caso deste trabalho (grafo estrela, com $E = V - 1$), o termo dominante reduz-se a $\Theta(V \log V)$, pois há poucas arestas a relaxar. Já no pior caso (grafo completo, com $E \approx V^2/2$), o tempo é $\Theta(V^2 \log V)$. A notação $\Theta((V + E) \log V)$ engloba todos esses cenários como limite superior comum. A complexidade de espaço é $\Theta(V + E)$, correspondente ao armazenamento da lista de adjacências, do vetor de distâncias e da fila de prioridade.

Esse comportamento teórico é exatamente o que se busca confirmar nos experimentos da Seção 4: espera-se que o tempo de execução cresça de acordo com $(V + E) \log V$, e que cenários com muitas arestas (grafos densos) sejam consideravelmente mais lentos que cenários com poucas arestas (grafos esparsos).

3 METODOLOGIA EXPERIMENTAL

3.1 Implementações

O algoritmo foi implementado de forma equivalente em duas linguagens: Python (arquivo `python/dijkstra.py`) e JavaScript executado no Node.js (arquivo `javascript/dijkstra.js`). Ambas as versões utilizam lista de adjacências para representar o grafo e fila de prioridade de mínimo para selecionar o próximo vértice. A escolha de duas linguagens permite comparar não apenas o comportamento assintótico — que deve ser o mesmo —, mas também o desempenho prático de cada ambiente de execução.

3.2 Cenários de teste e geração das entradas

Para avaliar o algoritmo de forma justa, foram definidos três cenários que representam o melhor caso, o caso médio e o pior caso em termos da quantidade de arestas processadas. As entradas (grafos) são geradas automaticamente com pesos aleatórios, usando uma semente fixa por rodada para garantir reprodutibilidade:

- Melhor caso — grafo estrela: todos os vértices se conectam apenas ao vértice de origem. O grafo possui o número mínimo de arestas ($E = n - 1$) e, após processar a origem, restam pouquíssimos relaxamentos. Representa o cenário mais favorável (grafo extremamente esparso).
- Caso médio — grafo aleatório esparsificado: cada par de vértices é conectado com probabilidade $p \sim 2 \ln(n)/n$. Isso produz um grafo conexo, porém esparso, representativo de muitas aplicações reais.
- Pior caso — grafo completo (denso): todos os pares de vértices estão conectados, resultando em $E \sim n(n-1)/2$ arestas. Esse cenário força o número máximo de relaxamentos e operações na fila de prioridade, sendo o mais custoso.

3.3 Tamanhos avaliados

Cada cenário foi executado em três tamanhos de grafo, definidos pelo número de vértices n : pequeno ($n = 300$), médio ($n = 1500$) e grande ($n = 4000$). A variação do tamanho permite observar a curva de crescimento do tempo de execução e compará-la com o modelo teórico $O((V + E) \log V)$.

3.4 Protocolo de medição

Cada combinação de cenário e tamanho foi executada 30 vezes (rodadas), totalizando 9 combinações por linguagem. Para cada rodada mediu-se o tempo de execução do algoritmo (sem contar a geração do grafo). A partir das 30 medições foram calculadas estatísticas descritivas: média, desvio padrão, valores mínimo e máximo,

além de métricas derivadas como coeficiente de variação (desvio/média), tempo por aresta e throughput (arestas processadas por segundo). A repetição em 30 rodadas reduz o efeito de ruídos do sistema operacional e do compilador JIT.

Para garantir a validade das medições, utilizaram-se funções de cronometragem de alta precisão, e não o relógio comum do sistema: em Python empregou-se `time.perf_counter()`, e em JavaScript a função `performance.now()` do módulo `perf_hooks` do Node.js. Ambas oferecem resolução de frações de microssegundo, adequada para medir execuções muito rápidas como as do melhor caso.

3.5 Ambiente de execução

Como o tempo de execução é sensível ao hardware e ao sistema operacional, todos os experimentos foram realizados na mesma máquina, cujas especificações são:

- Processador: Apple M4 (10 núcleos).
- Memória: 16 GB de RAM.
- Sistema operacional: macOS 26.3 (build 25D125).
- Interpretador Python: Python 3.9.6.
- Ambiente JavaScript: Node.js v24.2.0.

Para reduzir interferências externas e garantir medições confiáveis, adotou-se o isolamento dos processos: os benchmarks foram executados de forma isolada (um cenário por vez, em processo dedicado, sem outras aplicações pesadas concorrendo por CPU), e o tempo medido refere-se exclusivamente à execução do algoritmo, excluindo a etapa de geração do grafo. As 30 rodadas por cenário, combinadas a esse isolamento, atenuam o efeito de ruídos do escalonador do sistema operacional e do aquecimento do compilador JIT, tornando as estatísticas mais representativas.

3.6 Resumo dos cenários executados

As Tabelas 1, 2 e 3 consolidam, separadamente por caso (melhor, médio e pior), as estatísticas obtidas em cada tamanho e linguagem (média e desvio padrão das 30 rodadas, além dos valores mínimo e máximo). Elas servem de referência para a análise gráfica da seção seguinte.

Tabela 1 – Tempos de execução – Melhor caso (grafo estrela) (ms)

Linguagem	Tam.	n	Arestas	Média $\pm$ DP (ms)	Mín (ms)	Máx (ms)
Python	pequeno	300	149	0.087 ± 0.003	0.084	0.098
JavaScript	pequeno	300	299	0.138 ± 0.144	0.042	0.821
Python	medio	1500	749	0.486 ± 0.059	0.434	0.702
JavaScript	medio	1500	1499	0.272 ± 0.087	0.202	0.495
Python	grande	4000	1999	1.419 ± 0.059	1.322	1.558
JavaScript	grande	4000	3999	0.779 ± 0.218	0.557	1.497

Fonte: elaborado pelos autores (2026).

Tabela 2 – Tempos de execução – Caso médio (grafo aleatório esparsificado) (ms)

Linguagem	Tam.	n	Arestas	Média $\pm$ DP (ms)	Mín (ms)	Máx (ms)
Python	pequeno	300	1706	0.247 ± 0.014	0.227	0.286
JavaScript	pequeno	300	1651	0.093 ± 0.025	0.075	0.208
Python	medio	1500	10933	1.848 ± 0.077	1.715	2.091
JavaScript	medio	1500	11073	0.869 ± 0.347	0.583	2.478
Python	grande	4000	32834	6.989 ± 0.333	6.599	8.119
JavaScript	grande	4000	33318	3.631 ± 3.376	2.370	21.251

Fonte: elaborado pelos autores (2026).

Tabela 3 – Tempos de execução – Pior caso (grafo completo) (ms)

Linguagem	Tam.	n	Arestas	Média $\pm$ DP (ms)	Mín (ms)	Máx (ms)
Python	pequeno	300	44850	3.171 ± 0.363	2.917	4.652
JavaScript	pequeno	300	44850	0.988 ± 0.234	0.679	1.743
Python	medio	1500	1124250	181.487 ± 1.892	176.255	187.096
JavaScript	medio	1500	1124250	30.572 ± 8.539	16.258	59.872
Python	grande	4000	7998000	1725.937 ± 581.875	1340.103	3227.201
JavaScript	grande	4000	7998000	171.348 ± 46.340	126.084	329.141

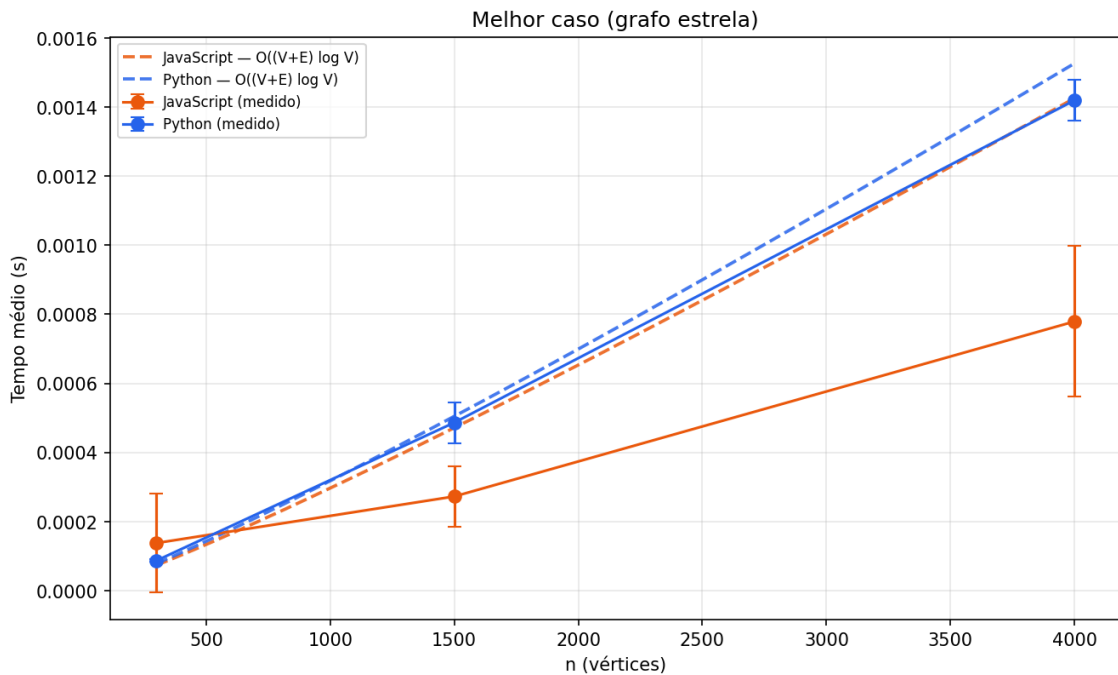
Fonte: elaborado pelos autores (2026).

4 RESULTADOS E ANÁLISE DOS GRÁFICOS

Esta seção apresenta e interpreta todos os gráficos gerados pelos experimentos, armazenados em `data/gráficos/`. Os gráficos estão organizados em quatro grupos: (i) curvas de tempo por cenário, (ii) dispersão e estabilidade das medições, (iii) comparação entre linguagens e (iv) validação do modelo teórico. Para cada figura é fornecida uma análise do que ela mostra.

4.1 Curvas de tempo por cenário

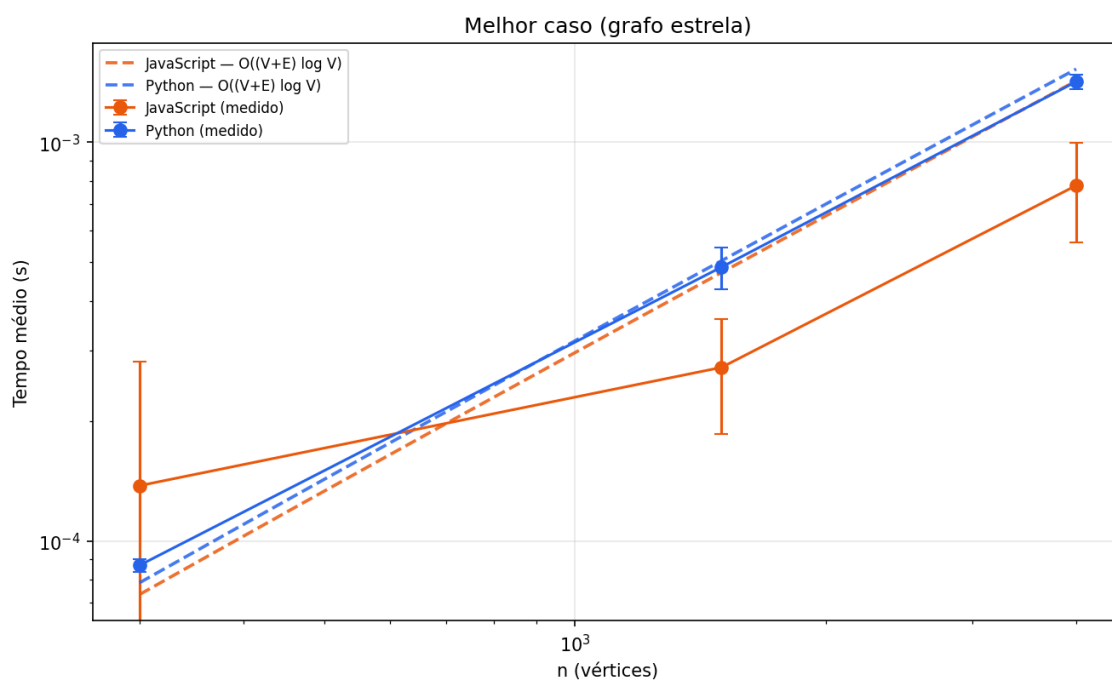
Figura 1 – Tempo no melhor caso (grafo estrela)



Fonte: elaborado pelos autores (2026).

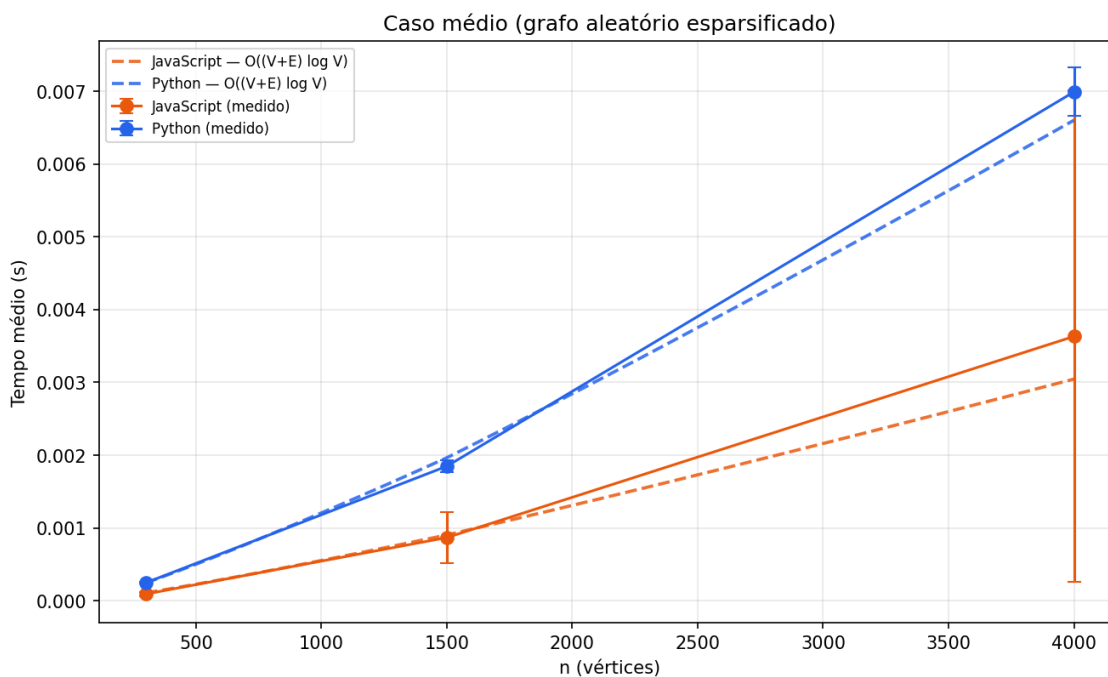
Neste cenário o grafo é extremamente esparsa ($E = n - 1$). Os tempos são muito baixos (frações de milissegundo) e crescem de forma quase linear com n , pois há poucas arestas a relaxar. A linha tracejada representa o modelo $O((V + E) \log V)$ ajustado por uma constante; observa-se boa aderência entre os pontos medidos e a curva teórica.

Figura 2 – Tempo no melhor caso em escala logarítmica



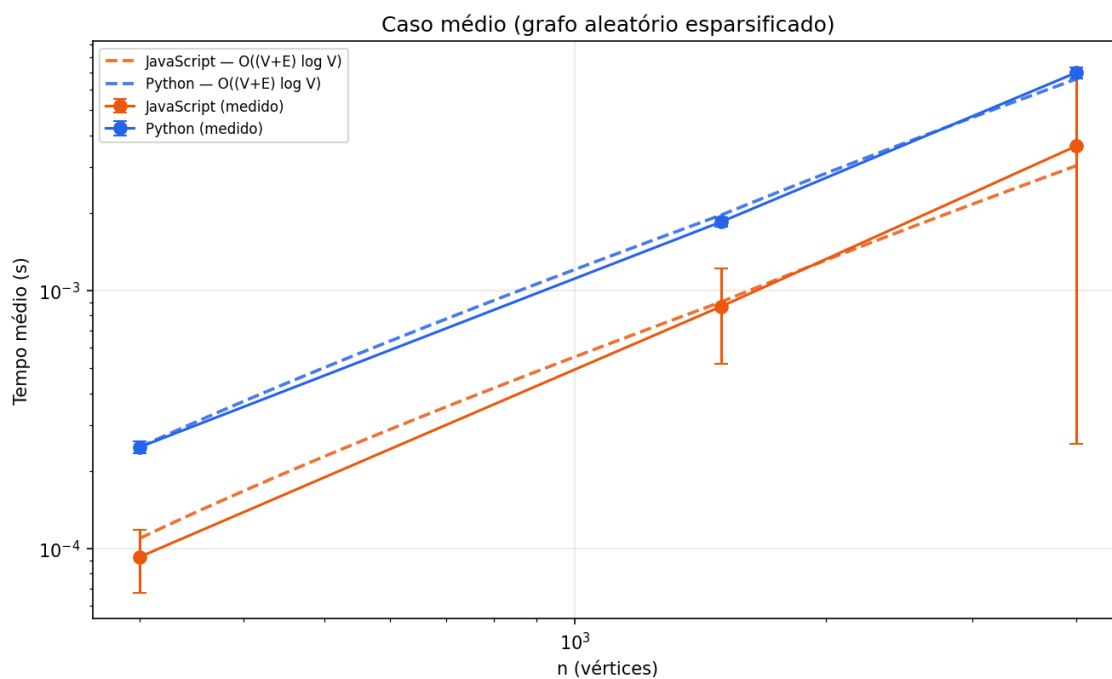
Fonte: elaborado pelos autores (2026).

Em escala log-log, uma função polinomial vira uma reta. A linearidade dos pontos confirma o crescimento controlado do tempo no melhor caso e facilita a comparação visual da inclinação entre Python e JavaScript.

Figura 3 – Tempo no caso médio (grafo aleatório esparsificado)

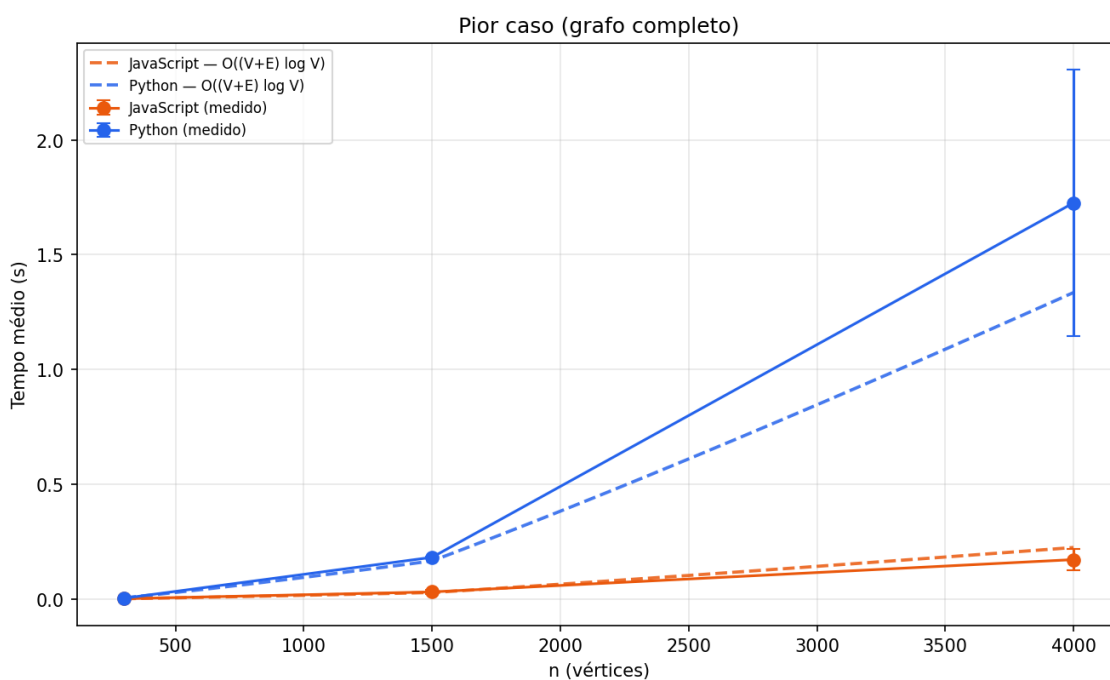
Fonte: elaborado pelos autores (2026).

Com o grafo esparsa porém mais conectado, o número de arestas cresce como $n \log n$. Os tempos permanecem na casa de poucos milissegundos e a curva acompanha o modelo teórico, evidenciando que o algoritmo escala bem em grafos representativos de aplicações reais.

Figura 4 – Tempo no caso médio em escala logarítmica

Fonte: elaborado pelos autores (2026).

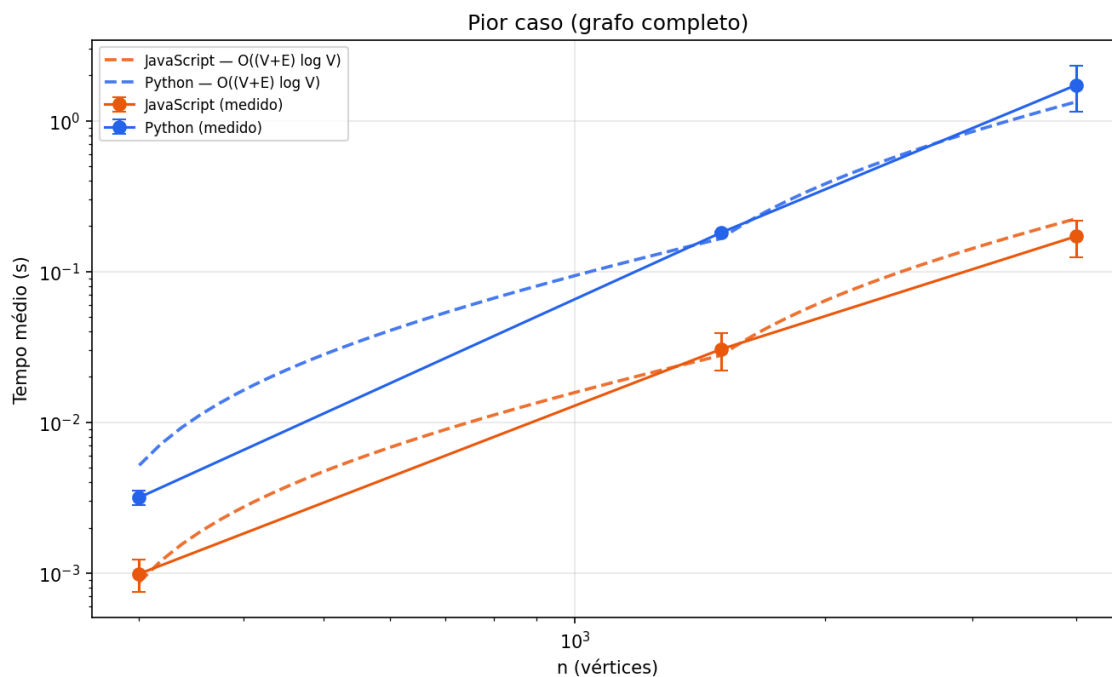
A versão logarítmica reforça a tendência polinomial suave do caso médio e mostra que a vantagem do JavaScript se mantém constante em diferentes tamanhos.

Figura 5 – Tempo no pior caso (grafo completo)

Fonte: elaborado pelos autores (2026).

Aqui o grafo é denso ($E \sim n^2/2$), o que torna o tempo muito mais elevado. Para $n = 4000$ o Python chega a cerca de 1,7 segundo, enquanto o JavaScript fica em torno de 0,17 segundo. O crescimento acentuado confirma que a quantidade de arestas é o fator dominante do custo.

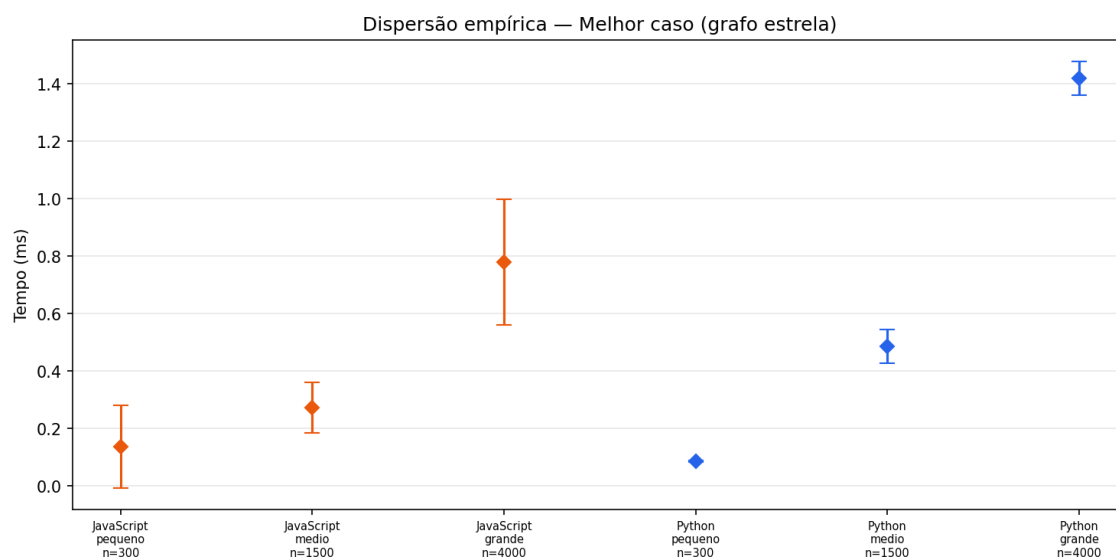
Figura 6 – Tempo no pior caso em escala logarítmica



Fonte: elaborado pelos autores (2026).

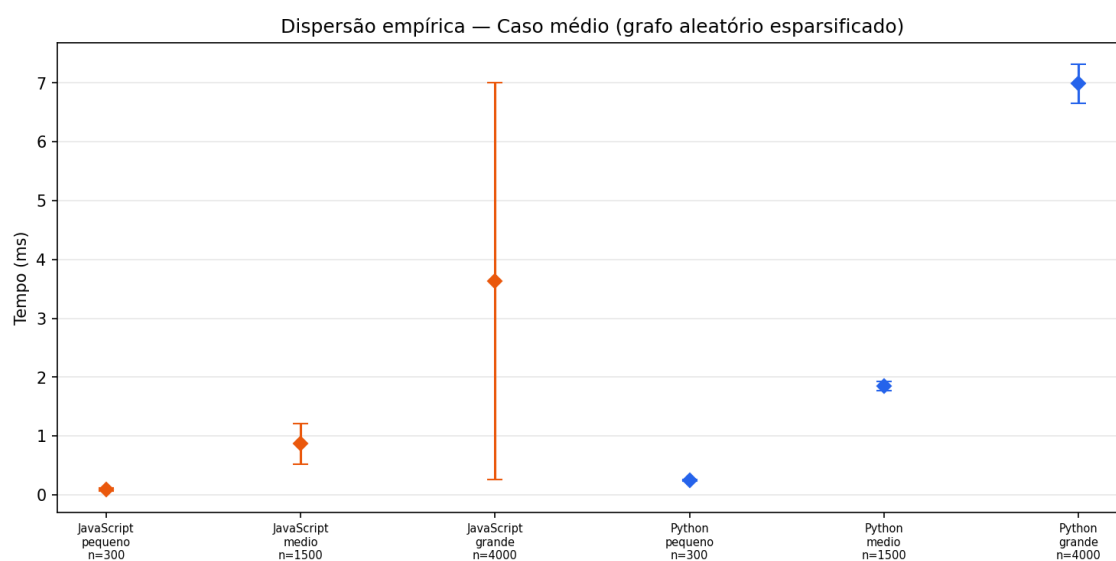
A escala log-log lineariza o crescimento e evidencia a inclinação íngreme característica de um grafo denso, na qual o termo $E \log V$ domina completamente o tempo de execução.

4.2 Dispersão e estabilidade das medições

Figura 7 – Dispersão empírica no melhor caso

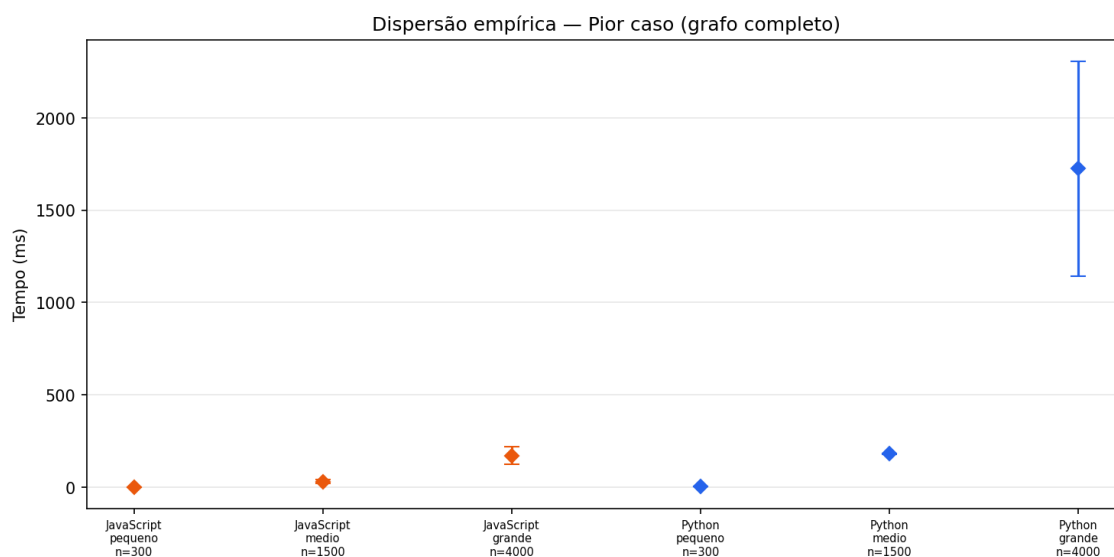
Fonte: elaborado pelos autores (2026).

Mostra o intervalo entre mínimo e máximo das 30 rodadas. No melhor caso, por envolver tempos minúsculos, a dispersão relativa é maior, especialmente em JavaScript, onde o aquecimento do compilador JIT influencia as primeiras medições.

Figura 8 – Dispersão empírica no caso médio

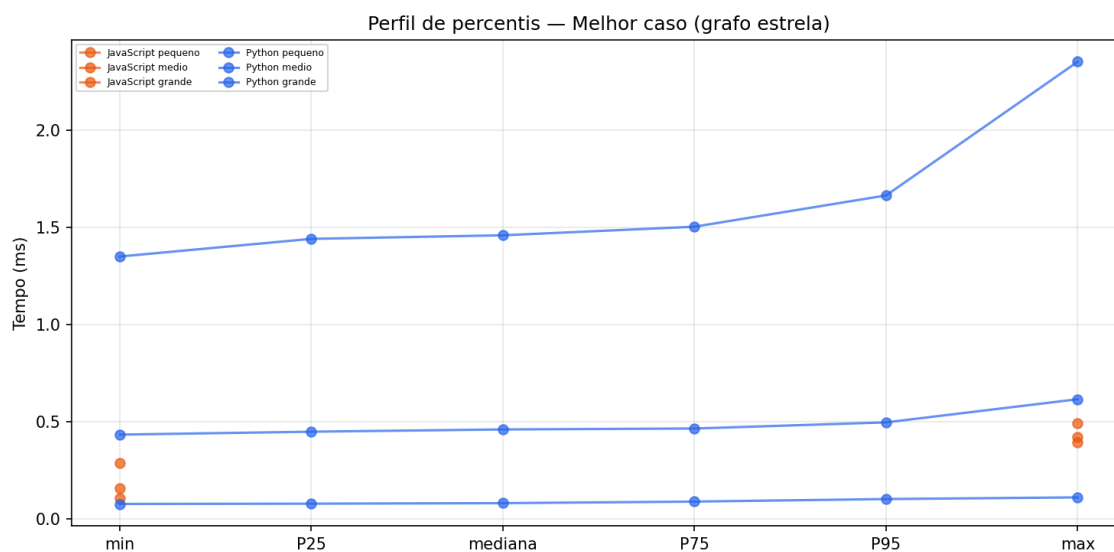
Fonte: elaborado pelos autores (2026).

A dispersão se reduz à medida que o trabalho por rodada aumenta, indicando medições mais estáveis no caso médio do que no melhor caso.

Figura 9 – Dispersão empírica no pior caso

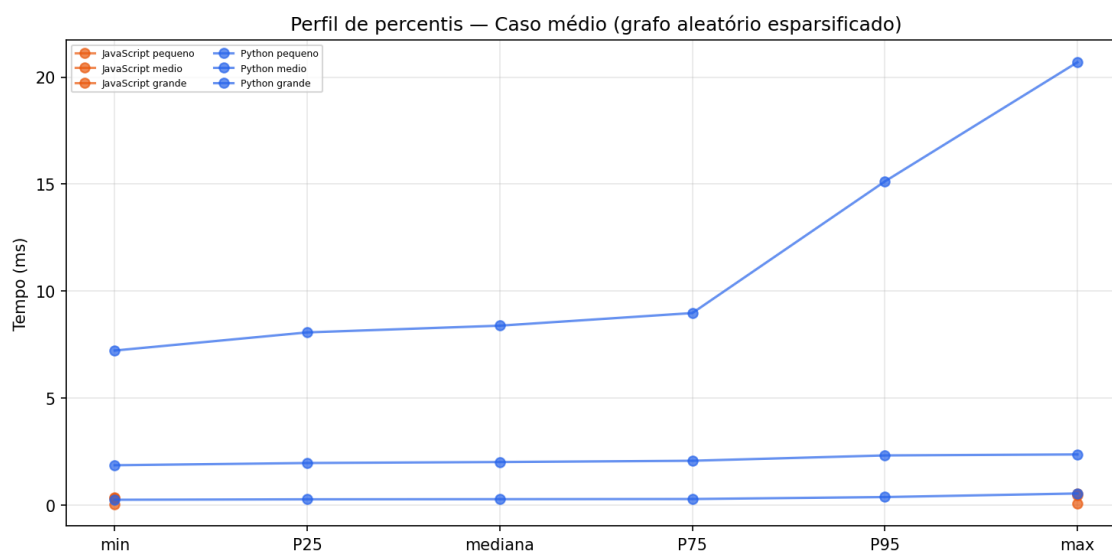
Fonte: elaborado pelos autores (2026).

No pior caso, embora os tempos absolutos sejam altos, a variação relativa é pequena (o algoritmo passa a maior parte do tempo em trabalho útil), o que torna as medições bastante consistentes.

Figura 10 – Perfil de percentis no melhor caso

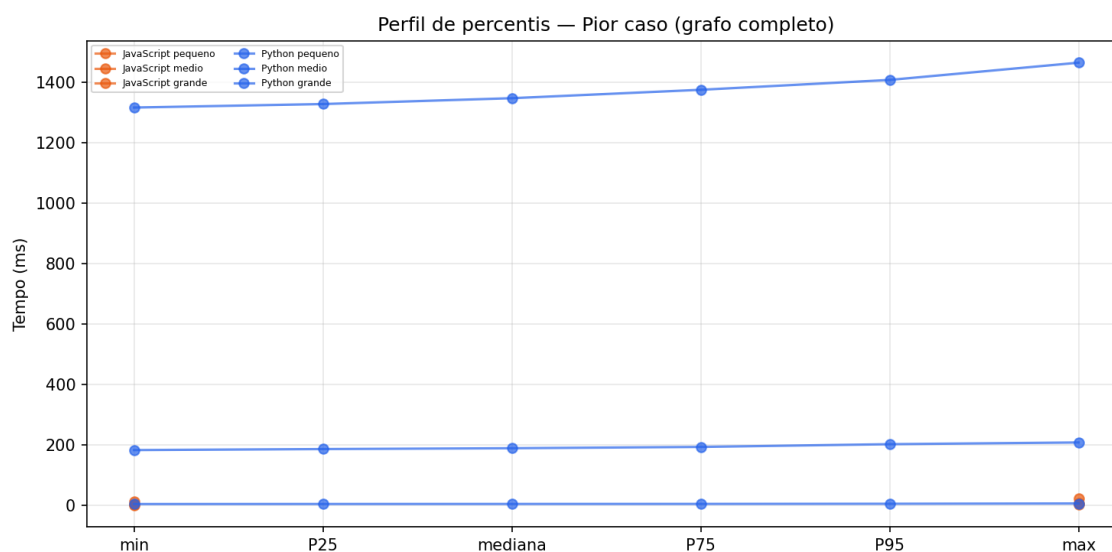
Fonte: elaborado pelos autores (2026).

Apresenta mínimo, P25, mediana, P75, P95 e máximo. A distância entre a mediana e o máximo revela a presença de valores atípicos (outliers) associados ao aquecimento inicial.

Figura 11 – Perfil de percentis no caso médio

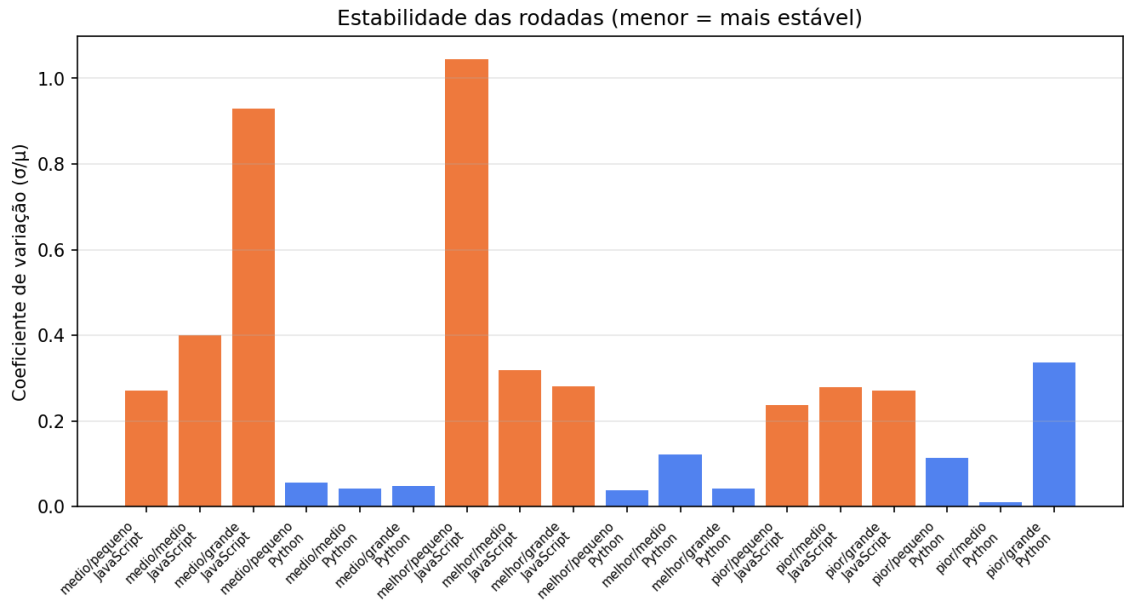
Fonte: elaborado pelos autores (2026).

Os percentis ficam próximos entre si, indicando uma distribuição concentrada e medições confiáveis.

Figura 12 – Perfil de percentis no pior caso

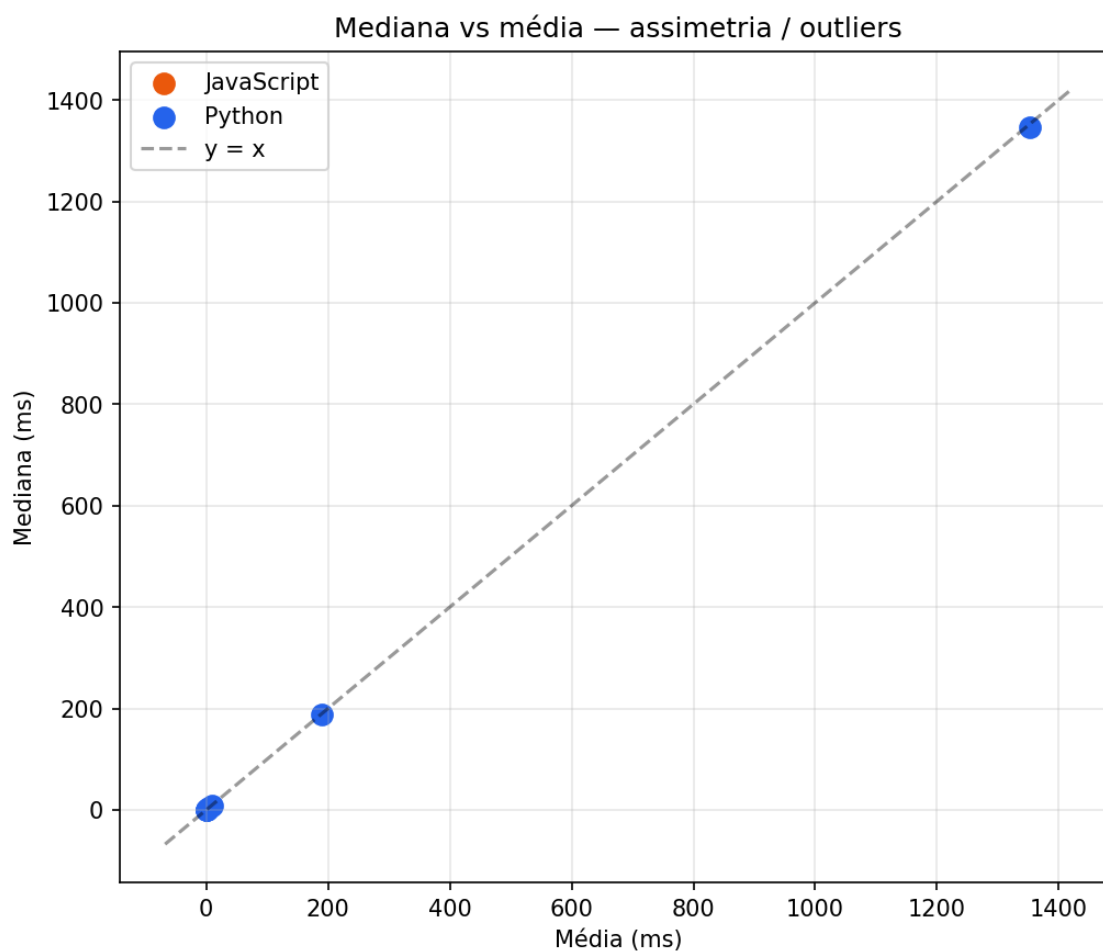
Fonte: elaborado pelos autores (2026).

A proximidade entre P25, mediana e P75 confirma a baixa dispersão relativa do pior caso, apesar dos tempos elevados.

Figura 13 – Coeficiente de variação por cenário

Fonte: elaborado pelos autores (2026).

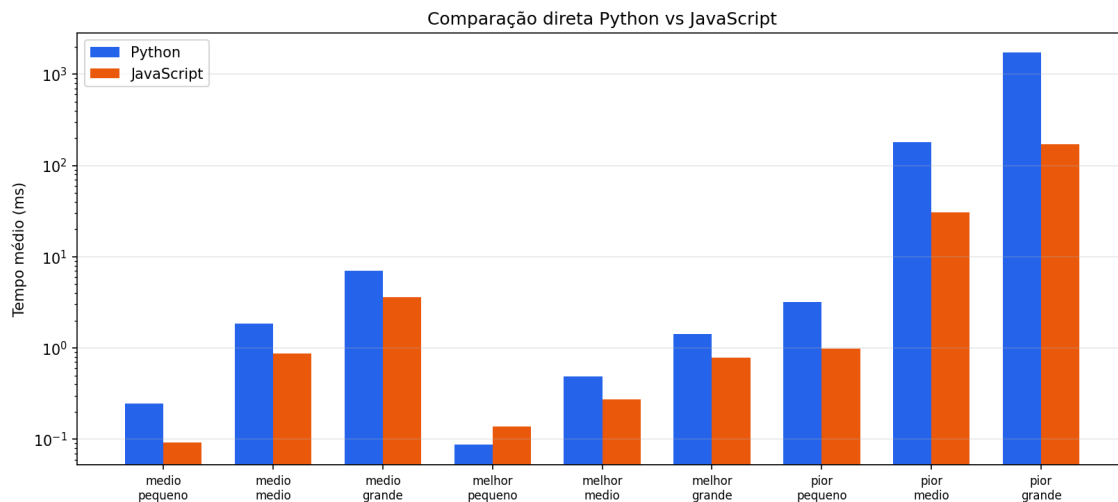
O coeficiente de variação (desvio/média) mede a estabilidade relativa. Barras menores indicam medições mais estáveis. Nota-se que o Python tende a ser mais estável que o JavaScript nos cenários pequenos, novamente por causa do JIT.

Figura 14 – Mediana versus média

Fonte: elaborado pelos autores (2026).

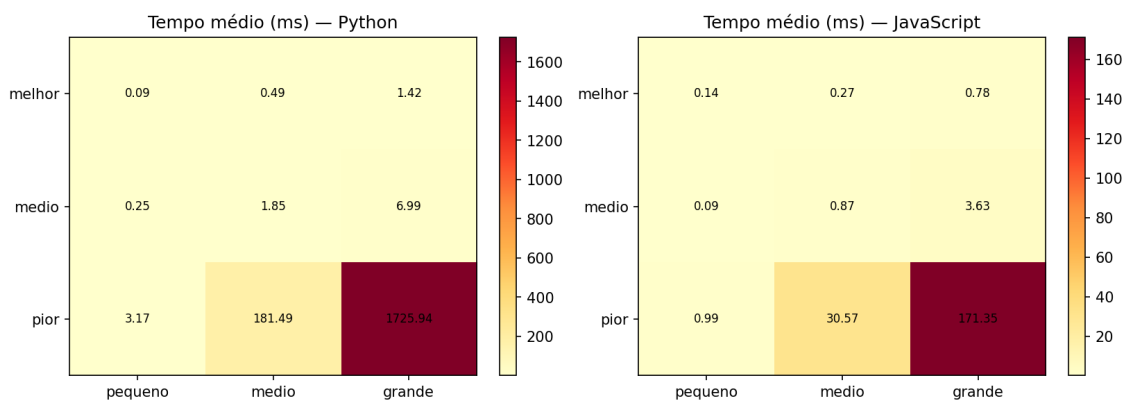
Quando os pontos ficam sobre a reta $y = x$, média e mediana coincidem e a distribuição é simétrica. Pontos acima da reta sugerem assimetria causada por outliers que elevam a média.

4.3 Comparação entre Python e JavaScript

Figura 15 – Comparação direta Python versus JavaScript

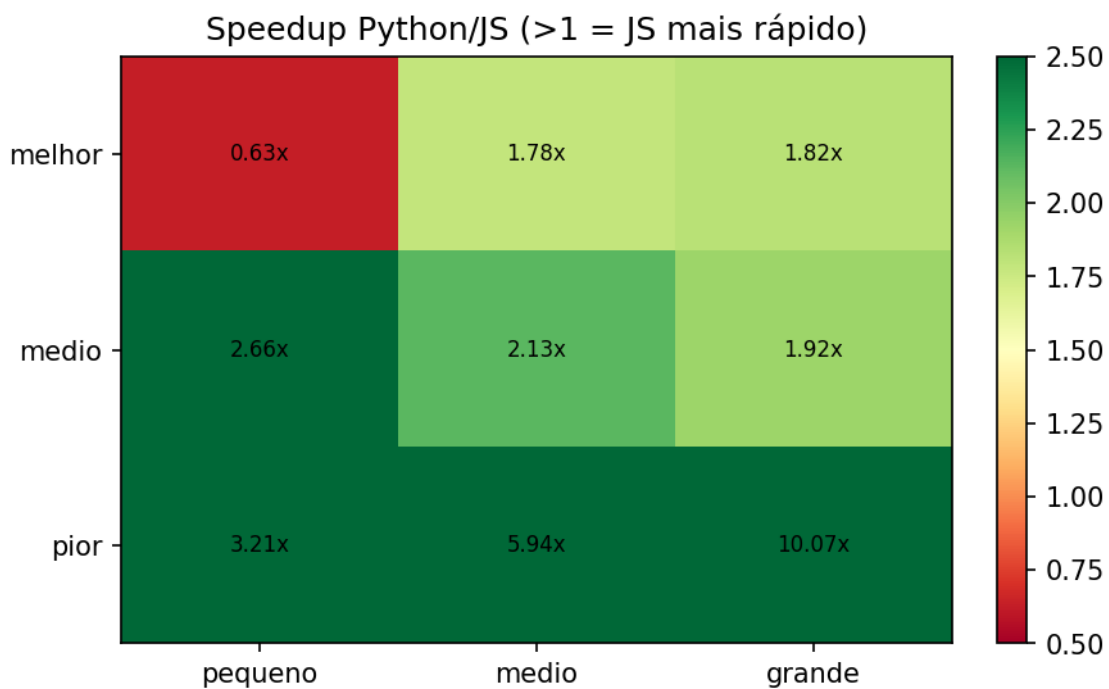
Fonte: elaborado pelos autores (2026).

Gráfico de barras (escala logarítmica) com o tempo médio lado a lado. Em praticamente todos os cenários o JavaScript (Node.js) é mais rápido, com vantagem expressiva no pior caso, onde chega a ser cerca de 10 vezes mais veloz que o Python.

Figura 16 – Mapa de calor dos tempos médios

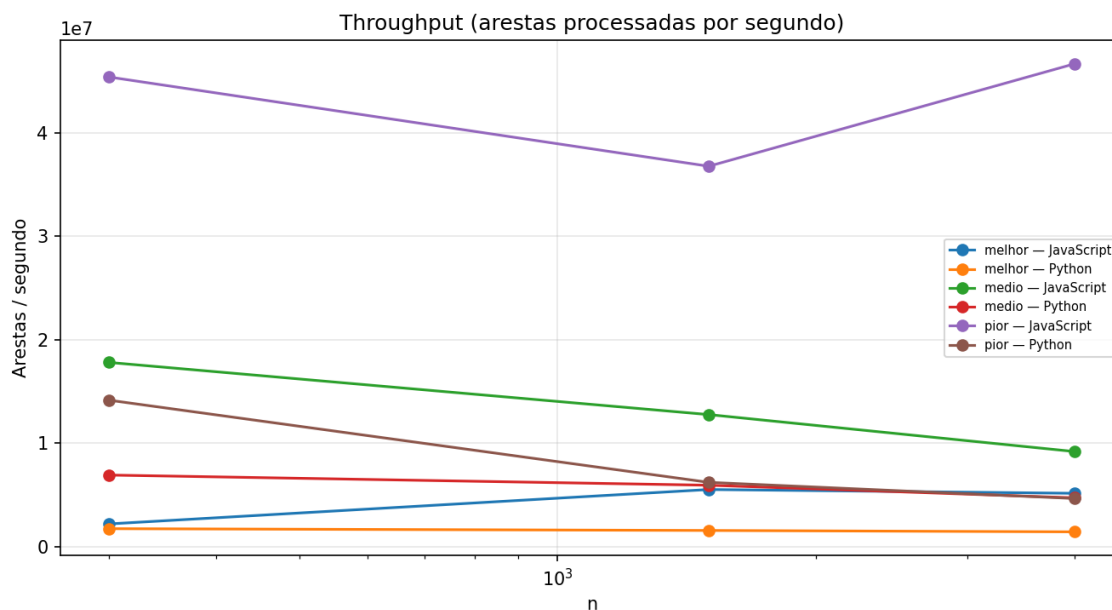
Fonte: elaborado pelos autores (2026).

Os mapas de calor resumem o tempo médio (ms) por caso e tamanho para cada linguagem. Tons mais escuros indicam tempos maiores, concentrados no pior caso com n grande.

Figura 17 – Mapa de calor do speedup (Python/JavaScript)

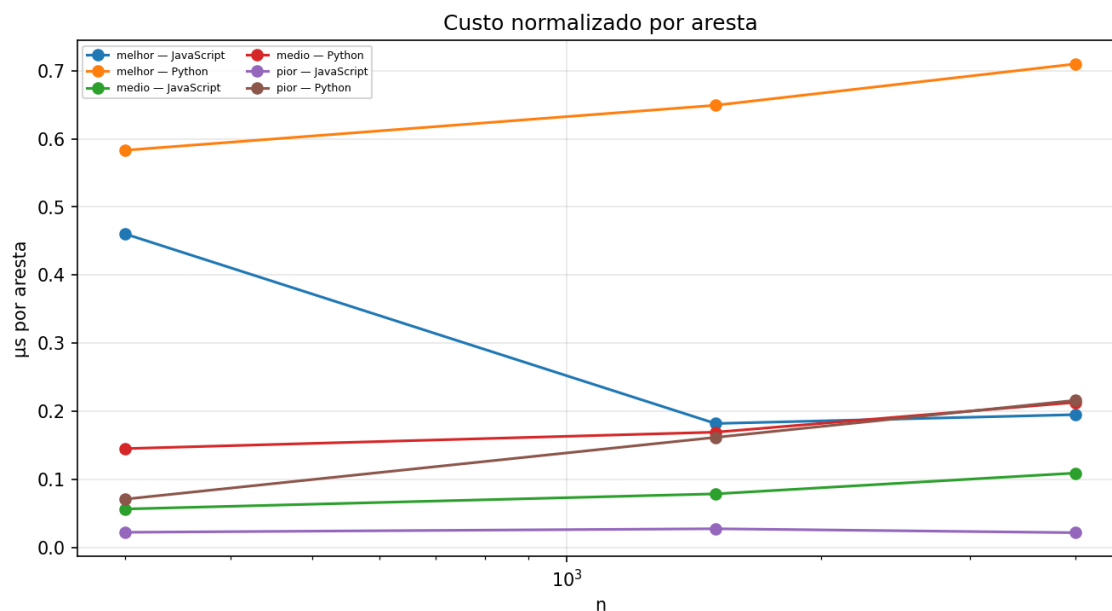
Fonte: elaborado pelos autores (2026).

Mostra a razão entre o tempo do Python e o do JavaScript. Valores acima de 1 significam que o JavaScript foi mais rápido. O ganho é maior nos grafos densos, onde o JIT do V8 otimiza muito bem os laços de relaxamento.

Figura 18 – Throughput (arestas processadas por segundo)

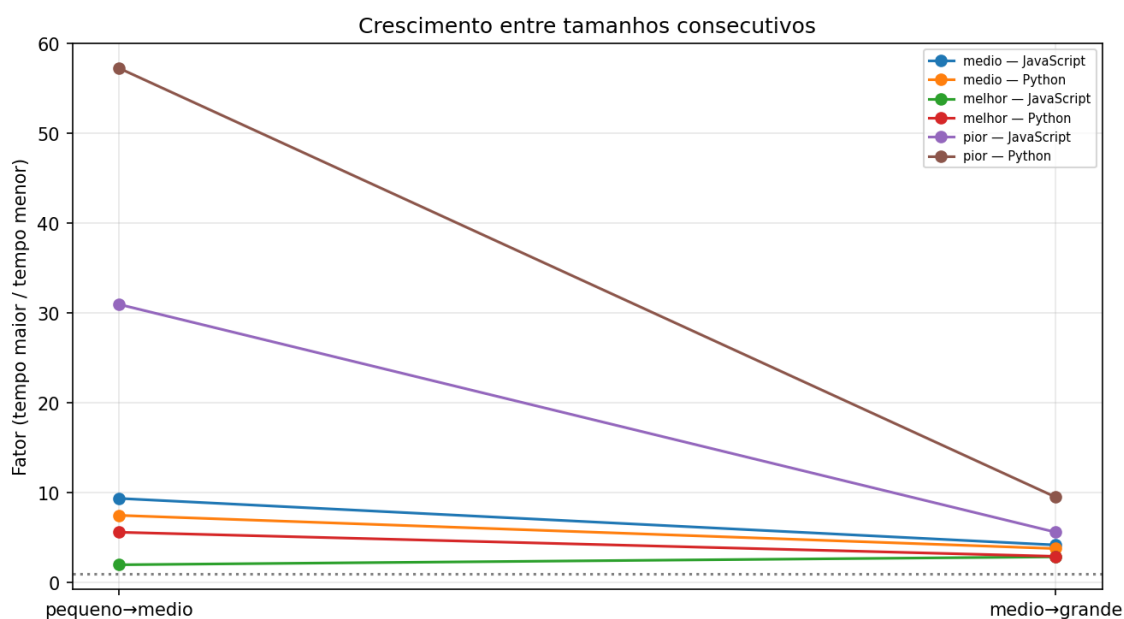
Fonte: elaborado pelos autores (2026).

Mede quantas arestas cada linguagem processa por segundo. O JavaScript apresenta throughput maior, sobretudo no pior caso, confirmando sua eficiência no processamento de grafos densos.

Figura 19 – Custo normalizado por aresta

Fonte: elaborado pelos autores (2026).

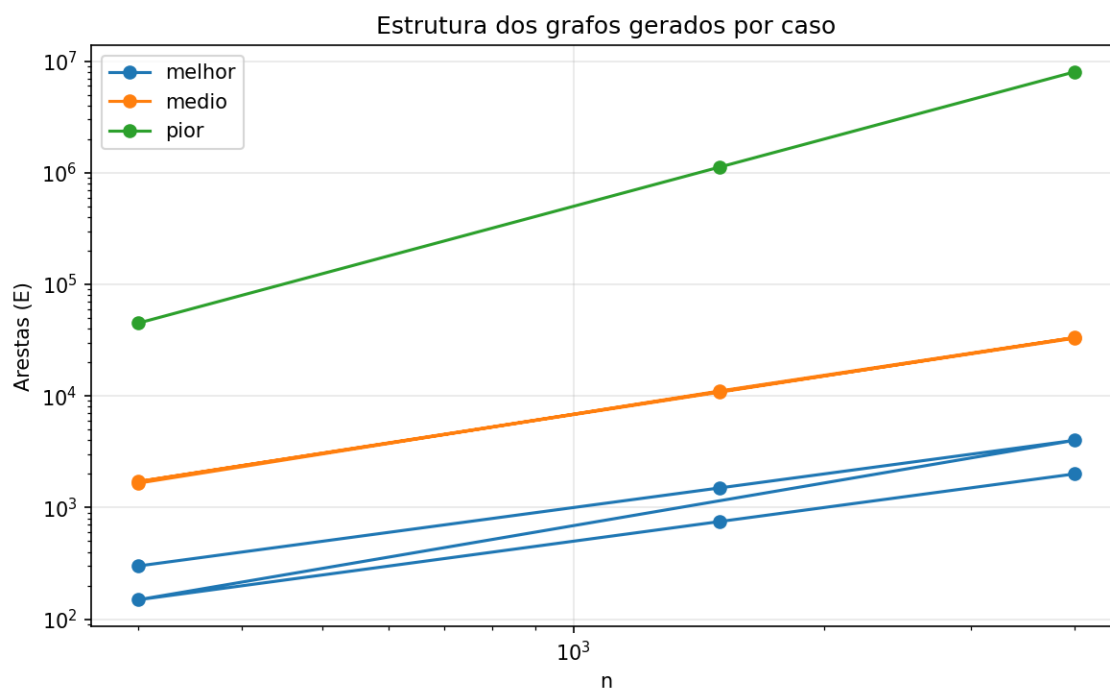
Divide o tempo total pelo número de arestas, isolando o custo unitário. Esse custo é praticamente constante dentro de cada cenário, o que é coerente com a complexidade linear-logarítmica do algoritmo.

Figura 20 – Fator de crescimento entre tamanhos consecutivos

Fonte: elaborado pelos autores (2026).

Mostra quantas vezes o tempo aumenta ao passar de um tamanho para o seguinte.
Fatores maiores no pior caso refletem o crescimento quadrático do número de arestas.

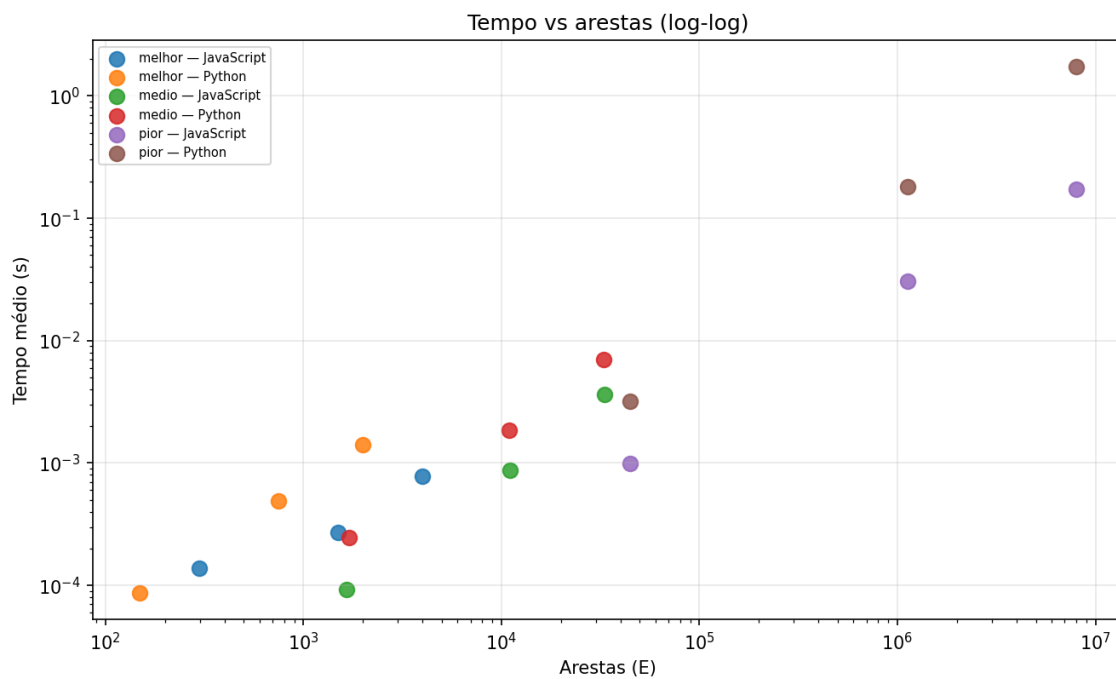
4.4 Validação do modelo teórico

Figura 21 – Estrutura dos grafos: arestas em função de n

Fonte: elaborado pelos autores (2026).

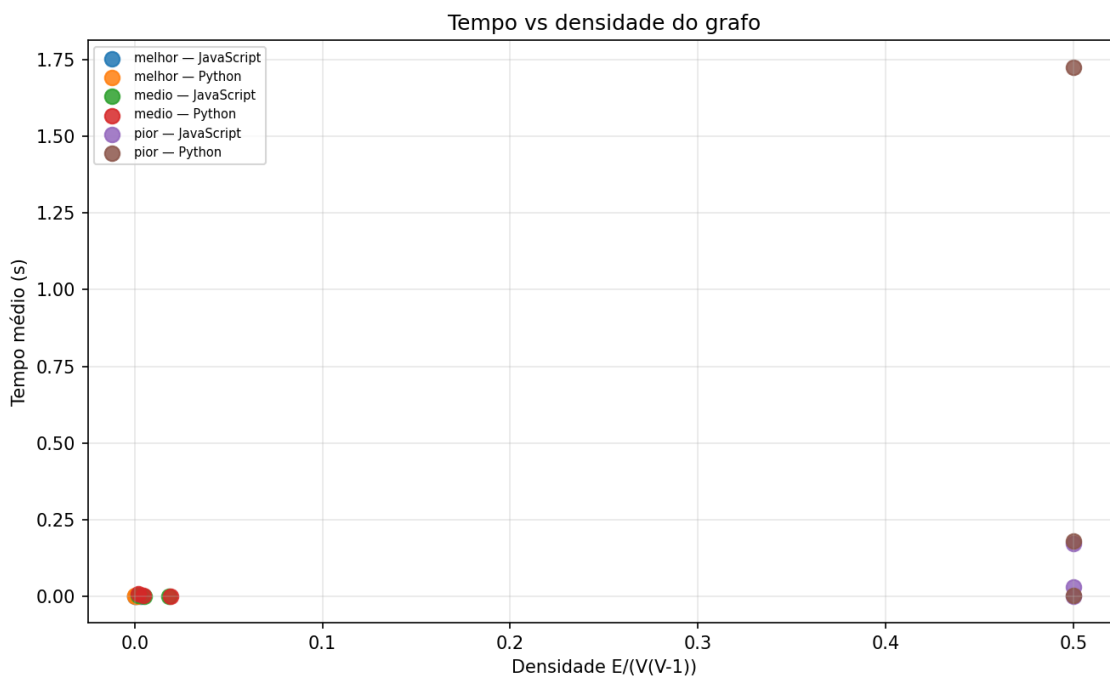
Confirma a natureza de cada cenário: o melhor caso cresce linearmente ($E \sim n$), o caso médio cresce como $n \log n$ e o pior caso cresce de forma quadrática ($E \sim n^2$), exatamente como projetado nos geradores de grafo.

Figura 22 – Tempo em função do número de arestas (log-log)



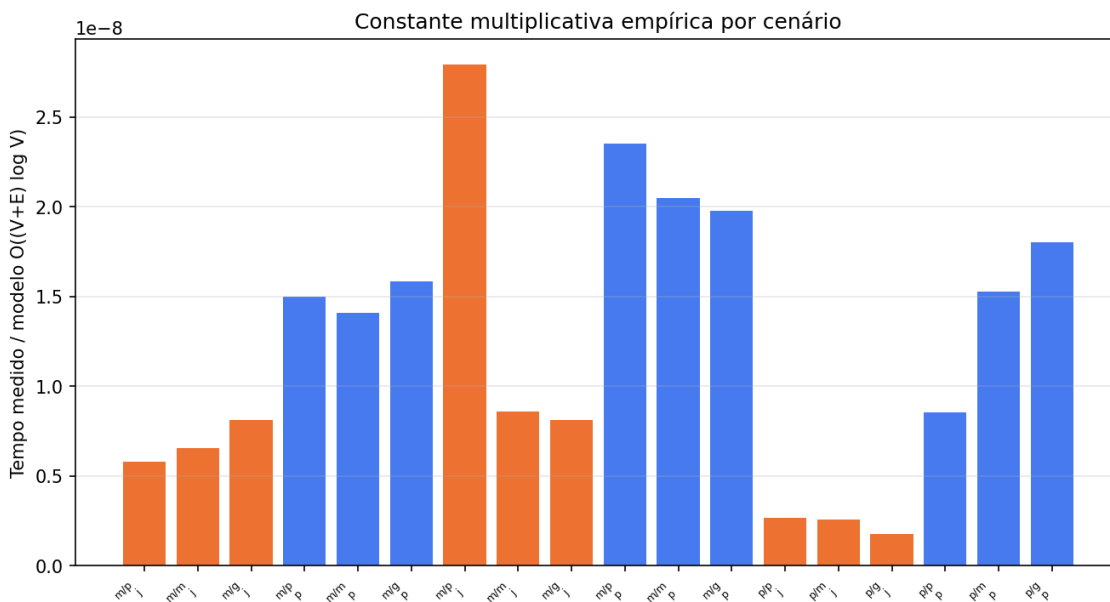
Fonte: elaborado pelos autores (2026).

Quando o tempo é colocado contra o número de arestas, os pontos dos três cenários se alinham, sugerindo que o número de arestas é o principal determinante do tempo de execução — em conformidade com o termo $E \log V$.

Figura 23 – Tempo em função da densidade do grafo

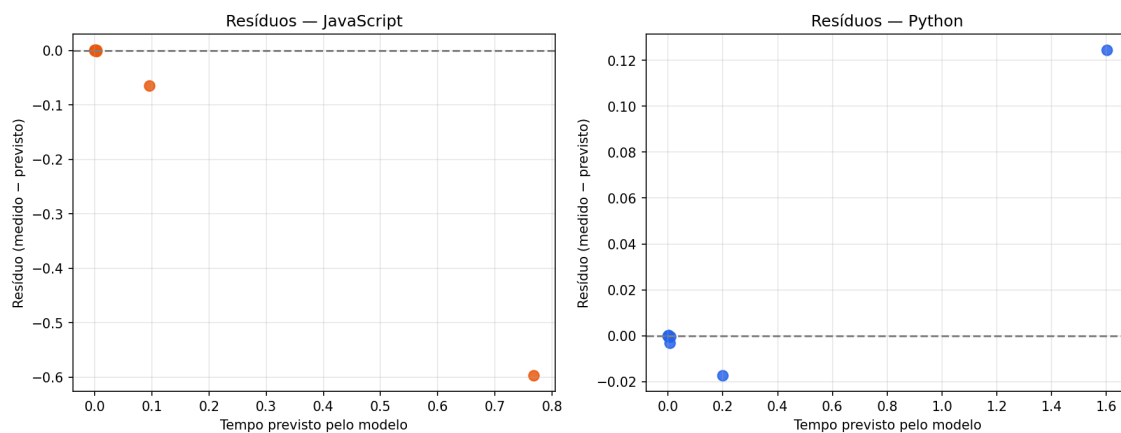
Fonte: elaborado pelos autores (2026).

Relaciona o tempo com a densidade (proporção de arestas presentes). Grafos mais densos consomem mais tempo, reforçando a influência de E .

Figura 24 – Constante multiplicativa empírica por cenário

Fonte: elaborado pelos autores (2026).

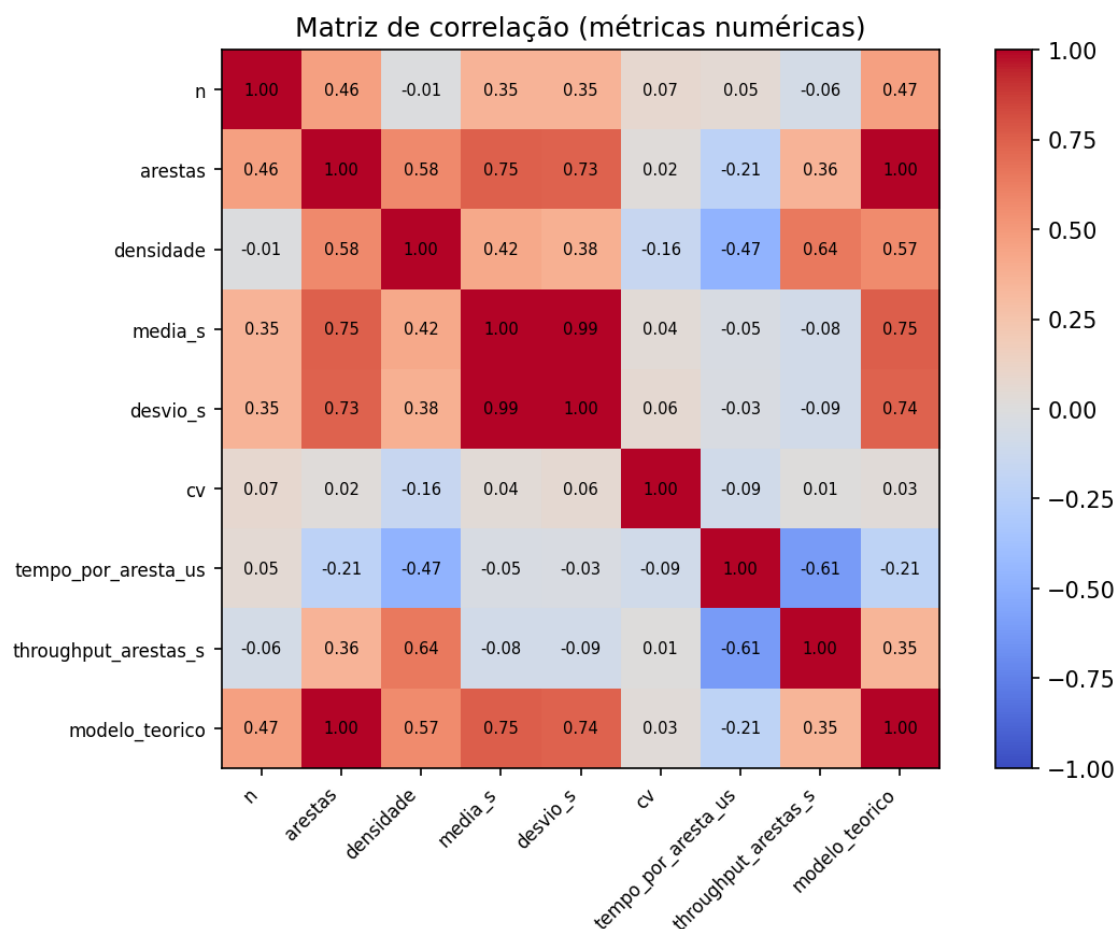
Razão entre o tempo medido e o valor do modelo $O((V + E) \log V)$. Se o modelo descreve bem a realidade, essa razão deve ser aproximadamente constante — e é o que se observa, com valores na mesma ordem de grandeza em todos os cenários.

Figura 25 – Resíduos do ajuste ao modelo teóricoAderência a $O((V+E) \log V)$ 

Fonte: elaborado pelos autores (2026).

Os resíduos (diferença entre tempo medido e previsto) distribuem-se em torno de zero sem padrão sistemático evidente, indicando boa aderência do algoritmo à complexidade $O((V + E) \log V)$.

Figura 26 – Matriz de correlação das métricas

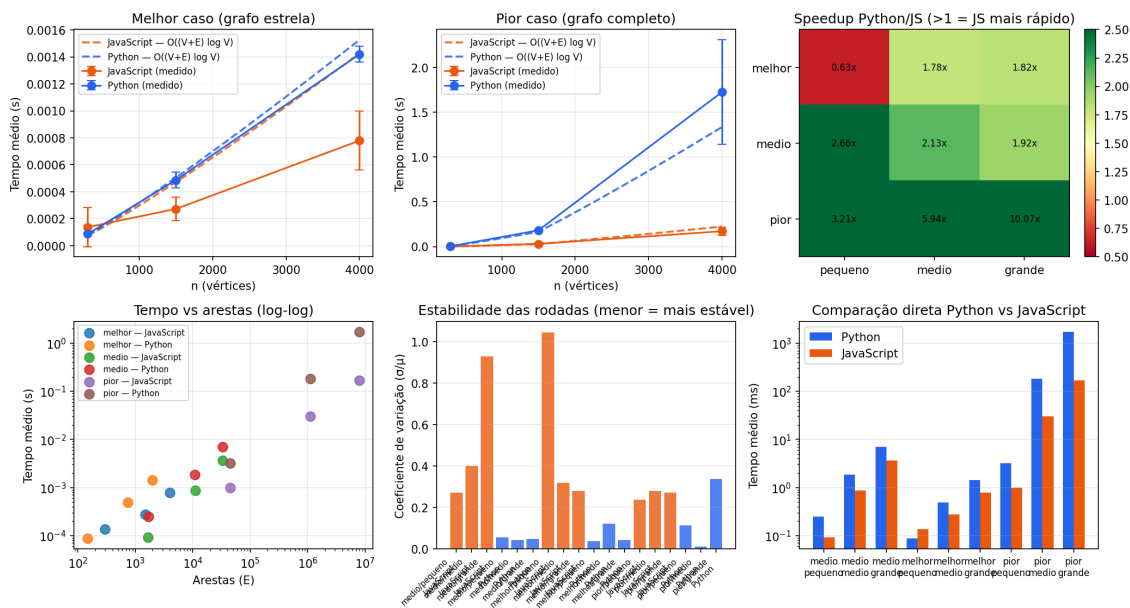


Fonte: elaborado pelos autores (2026).

Correlação entre as variáveis numéricas. Observa-se forte correlação positiva entre número de arestas, modelo teórico e tempo médio, confirmando que essas grandezas variam juntas.

Figura 27 – Painel-resumo dos experimentos

Painel resumo — Dijkstra Python vs JavaScript



Fonte: elaborado pelos autores (2026).

Painel consolidado com os principais gráficos em uma única visão, útil para uma leitura rápida e comparativa de todo o experimento.

5 APLICABILIDADE: EFICIÊNCIA E LIMITAÇÕES

Além do estudo de desempenho, é importante discutir em quais contextos o algoritmo de Dijkstra é eficiente e quais são as suas limitações práticas.

5.1 Contextos de eficiência e aplicações

O algoritmo de Dijkstra é a escolha natural para o problema do caminho mínimo de origem única em grafos com pesos não negativos, especialmente quando os grafos são esparsos. Como o experimento evidenciou, em grafos esparsos (melhor caso e caso médio) o tempo permanece na ordem de poucos milissegundos mesmo para milhares de vértices, o que o torna adequado para aplicações de tempo quase real. Entre os principais contextos de uso estão:

- Sistemas de navegação e mapas (GPS): cálculo da rota mais curta ou mais rápida entre dois pontos de uma malha viária, que é tipicamente esparsa.
- Redes de computadores: protocolos de roteamento de estado de enlace, como o OSPF, usam Dijkstra para determinar os melhores caminhos entre roteadores.
- Logística e transporte: planejamento de entregas e otimização de trajetos com menor custo, distância ou tempo.
- Jogos e inteligência artificial: movimentação de personagens e busca de caminhos em mapas, frequentemente como base do algoritmo A*.
- Análise de redes sociais e telecomunicações: medição de proximidade e centralidade entre nós.

5.2 Limitações

Apesar da ampla aplicabilidade, o algoritmo possui limitações relevantes que devem ser observadas:

- Pesos negativos: Dijkstra não funciona corretamente quando há arestas com peso negativo, pois sua estratégia gulosa pressupõe que fixar a menor distância de um vértice é definitivo. Nesses casos deve-se usar o algoritmo de Bellman-Ford, de complexidade $O(V \cdot E)$.
- Grafos densos: como mostrado no pior caso, em grafos completos ($E \approx V^2$) o tempo cresce de forma acentuada ($\Theta(V^2 \log V)$), tornando-se custoso para grafos muito grandes e densos.
- Origem única: cada execução calcula as distâncias a partir de uma única origem. Para todos os pares de vértices, pode ser mais vantajoso usar o algoritmo de Floyd-Warshall, de complexidade $O(V^3)$.
- Memória: o armazenamento da lista de adjacências exige espaço $\Theta(V + E)$, o que pode ser proibitivo em grafos densos de grande porte.

Em resumo, Dijkstra é altamente eficiente para grafos esparsos com pesos não negativos, mas existem cenários — pesos negativos, necessidade de todos os pares ou grafos extremamente densos — em que outras abordagens são preferíveis.

6 REFLEXÃO TEÓRICA: CLASSES DE COMPLEXIDADE

Esta seção relaciona o algoritmo estudado com as classes de complexidade computacional, em especial as classes P e NP, e discute a existência de problemas correlatos de difícil solução.

6.1 O problema pertence à classe P?

Sim. A classe P é composta pelos problemas de decisão que podem ser resolvidos por um algoritmo determinístico em tempo polinomial em relação ao tamanho da entrada. O problema do caminho mínimo resolvido por Dijkstra tem complexidade $O((V + E) \log V)$, que é polinomial. Portanto, o problema pertence à classe P: existe um algoritmo eficiente (de tempo polinomial) que o resolve de forma exata. Por consequência, ele também pertence à classe NP, já que todo problema da classe P também está contido na classe NP.

6.2 Existe uma versão NP do problema?

O problema do caminho mínimo, na sua forma usual, não é NP-difícil — ele é eficientemente resolvível. Contudo, pequenas variações no enunciado podem torná-lo intratável (NP-difícil). Dois exemplos clássicos:

- Caminho mais longo (longest path): encontrar o caminho simples de maior peso entre dois vértices é um problema NP-difícil, em forte contraste com o caminho mínimo, que está em P.
- Caminho com restrições adicionais: o problema do caminho mínimo com restrição de recursos (Resource Constrained Shortest Path), em que cada aresta consome um recurso limitado, é NP-difícil.

Ou seja, a fronteira entre o tratável e o intratável é tênue: trocar 'menor' por 'maior', ou adicionar restrições, pode mudar a classe de complexidade do problema.

6.3 Problemas semelhantes NP-completos

Existem problemas de grafos com pesos que se assemelham ao caminho mínimo, mas que são NP-completos (na forma de decisão) ou NP-difíceis (na forma de otimização). O mais emblemático é:

- Problema do Caixeiro-Viajante (Travelling Salesman Problem - TSP): encontrar o ciclo de menor custo que visita todos os vértices exatamente uma vez. Sua versão de decisão é NP-completa. Embora também trate de minimizar custo em um grafo

ponderado, sua dificuldade é muito superior à do caminho mínimo.

- Caminho Hamiltoniano: determinar se existe um caminho que passa por todos os vértices exatamente uma vez. É NP-completo e está diretamente relacionado ao TSP.

A comparação é instrutiva: enquanto o caminho mínimo (Dijkstra) está em P e escala bem, o TSP e o caminho hamiltoniano não possuem, até hoje, algoritmo de tempo polinomial conhecido. Caso fosse descoberto um algoritmo polinomial para qualquer problema NP-completo, ter-se-ia $P = NP$ — uma das maiores questões em aberto da Ciência da Computação. O fato de Dijkstra pertencer a P é, portanto, uma vantagem teórica expressiva: garante eficiência comprovada, ao contrário dos problemas NP-completos correlatos.

7 EXECUÇÃO DO PROJETO

Esta seção descreve como reproduzir todo o experimento a partir do código-fonte. A estrutura de pastas do projeto é a seguinte:

python/	codigo + benchmark em Python
javascript/	codigo + benchmark em JavaScript (Node.js)
data/	CSVs e graficos gerados pelos experimentos
notebooks/	analise interativa e gerador do relatorio
requirements.txt	dependencias Python
package.json	atalho npm para o benchmark JavaScript

7.1 Pré-requisitos

É necessário ter Python 3 e Node.js instalados. As dependências de análise em Python (matplotlib, numpy, pandas e jupyter) são instaladas com o comando:

```
pip3 install -r requirements.txt
```

7.2 Executando os benchmarks

Os benchmarks executam o algoritmo 30 vezes em cada cenário e salvam os resultados em arquivos CSV dentro da pasta data/. Para rodar a versão Python e a versão JavaScript:

```
# Benchmark em Python (gera data/resultados_python.csv)
python3 python/benchmark.py

# Benchmark em JavaScript (gera data/resultados_javascript.csv)
npm run benchmark
```

Para um teste rápido, com menos rodadas e grafos menores, utilize as variantes:

```
python3 python/benchmark.py --quick
npm run benchmark:quick
```

7.3 Gerando a análise e os gráficos

A análise interativa, que produz todos os gráficos discutidos na Seção 4, está no notebook Jupyter. Para abri-lo:

```
jupyter notebook notebooks/analise_dijkstra.ipynb
```

7.4 Gerando este relatório em PDF

Por fim, este próprio relatório em PDF, no padrão ABNT e com todos os gráficos explicados, é gerado pelo comando abaixo. A saída é o arquivo data/relatorio_dijkstra_abnt.pdf:

```
python3 notebooks/gerar_relatorio_abnt.py
```

8 CONCLUSÃO

Este trabalho apresentou um estudo teórico e experimental do algoritmo de Dijkstra para o problema do caminho mínimo. Do ponto de vista teórico, mostrou-se que o algoritmo, implementado com fila de prioridade baseada em heap binário, resolve o problema com limite superior $O((V + E) \log V)$, limite inferior $\Omega(V + E)$ e limite justo $\Theta((V + E) \log V)$ nos grafos densos, sendo o número de arestas o fator que mais influencia o custo. Por ter solução em tempo polinomial, o problema pertence à classe P, ao contrário de problemas correlatos como o Caixeiro-Viajante, que é NP-completo.

Os experimentos confirmaram a previsão teórica: o tempo de execução cresce de acordo com $(V + E) \log V$ em todos os cenários, e os grafos densos (pior caso) são ordens de grandeza mais lentos que os grafos esparsos (melhor caso). A análise de resíduos e da razão entre tempo medido e modelo teórico evidenciou boa aderência do comportamento empírico à complexidade esperada.

Na comparação entre linguagens, o JavaScript executado no Node.js mostrou-se consistentemente mais rápido que o Python, com vantagem expressiva nos grafos densos (cerca de dez vezes no pior caso de maior tamanho), graças às otimizações do compilador JIT do motor V8. O Python, por sua vez, apresentou medições mais estáveis nos cenários menores. Ambas as implementações, contudo, exibiram a mesma curva de crescimento assintótico, o que reforça que a complexidade é uma propriedade do algoritmo, e não da linguagem.

REFERÊNCIAS

CORMEN, Thomas H. et al. Algoritmos: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

DIJKSTRA, Edsger W. A note on two problems in connexion with graphs. *Numerische Mathematik*, v. 1, p. 269-271, 1959.

SEEDGEWICK, Robert; WAYNE, Kevin. Algorithms. 4. ed. Boston: Addison-Wesley, 2011.

ZIVIANI, Nivio. Projeto de algoritmos com implementações em Java e C++. São Paulo: Cengage Learning, 2007.